

2026-05-06-p2-f22-aider-git-auto-commit-design

Phase 2 / Feature 22 — Aider Git Auto-Commit Per Change

Date: 2026-05-06 **Status:** Approved (auto-approved per programme cadence) **Programme:** CLI-Agent Fusion — Phase 2 port (codex / cursor / aider patterns)

Programme position: F22 is the **second** Phase 2 feature. T01 (bootstrap) opens the F22 evidence section in `docs/improvements/07_phase_2_evidence.md` (created in F21); T09 (close-out) records F22's runtime evidence beneath F21's.

1. Goal

Ship a real, end-to-end **per-edit git auto-commit** facility for the HelixCode CLI agent, modelled verbatim on **aider** (`cli_agents/aider/`) so that every accepted file-mutating tool call (e.g. `fs_edit`, `fs_write`, `smart_edit`, `multiedit_commit`, `notebook_edit`) results in a real `git commit` against the live working tree, with a real LLM-summarised message, a real Co-Authored-By: HelixCode <noreply@helixcode.dev> trailer, and a real working-tree-clean post-condition. The feature is **default-on** (Q4=A) so users immediately see commit history materialise as the agent works; opt-out is available at three levels (`env HELIXCODE_GIT_AUTO_COMMIT=off`, `runtime /git_auto_commit off`, `per-edit arg _helix_skip_git_commit:true`). One commit per accepted edit (Q1=A) — aider's default cadence — so the resulting history is a faithful step-by-step trace of the agent's work, reviewable, revertable, and `git bisect`-able. The commit message is **LLM-summarised from the actual diff** (Q2=A) using the same `llm.Provider` already wired in `cmd/cli/main.go`; a deterministic fallback handles LLM unavailability so a flaky upstream NEVER blocks a commit. The co-author trailer (Q3=A) is appended to every auto-commit unconditionally; the user surface for runtime control is the `/git_auto_commit slash` command (Q5=A) with subcommands `status / on / off / show`. NO cobra subcommand.

Three concrete user surfaces ship together:

1. **autocommit package** (`helix_code/internal/autocommit/`) — `AutoCommitter` (encapsulates the live state: enabled flag stored in `atomic.Bool`, `llm.Provider` reference, git working-directory path, logger), `CommitContext` (per-call value: tool name, args, mutated paths, captured-before snapshot of the diff), `MessageSummariser` (LLM-driven; deterministic fallback when LLM unavailable), `Git` (thin wrapper over `os/exec` for `git status / git diff / git add / git commit / git rev-parse --is-inside-work-tree`), sentinel errors, env-var helper. The committer is constructed once at startup with the initial enabled state resolved from `HELIXCODE_GIT_AUTO_COMMIT` env var (default on — anything other than the literal string `off` is on, so typos default to safe-on). Public API: `MaybeCommit(ctx, CommitContext) (CommitResult, error)`, `SetEnabled(bool)`, `Enabled() bool`, `IsGitRepo() bool`.

2. **Registry post-Execute hook** — `internal/tools/registry.go::Execute` already has a hook point AFTER `tool.Execute` succeeds (the F13 LSP-auto-trigger fires there). F22 adds a SECOND post-Execute hook adjacent to it: `r.fireAutoCommit(ctx, name, params, result, execErr)` invoked ONLY when `execErr == nil` AND the F21 approval gate previously returned `Allow` (or `Prompt` then user-approved) AND the tool's `RequiresApproval()` level is `LevelEdit` or `LevelAll` (the file-mutating tiers; `LevelReadOnly` and `LevelRun` are EXCLUDED — read tools don't touch files; shell tools may touch files but the user is already supervising the shell). The hook detects “did this call actually mutate a tracked path” via `git status --porcelain` (positive evidence: dirty tracked path → commit; clean tracked path → no-op, even though the tool reported success). Per-edit opt-out: if `params["_helix_skip_git_commit"] == true`, the hook returns immediately without calling `MaybeCommit`. The hook NEVER propagates auto-commit errors back to the caller (commit failure is logged at WARN, the tool result is returned verbatim — auto-commit is a best-effort observability layer, not a blocking gate).
3. `/git_auto_commit slash command + env var (Q5=A)` — `HELIXCODE_GIT_AUTO_COMMIT=off` env var (any non-off value, including unset, is on); `/git_auto_commit slash command` with four subcommands: `/git_auto_commit status` (active state + git-repo-availability + provider summary), `/git_auto_commit on` (mutates the committer's atomic flag), `/git_auto_commit off` (mutates the flag), `/git_auto_commit show` (prints the commit message template + co-author trailer for documentation purposes). NO cobra subcommand.

The single largest bluff vector for F22 is “**commit succeeded but working tree still dirty**” — the auto-committer reports “committed” but `git status` still shows untracked or modified files, because `git add` selected the wrong path (e.g. used the tool's reported path which doesn't match the actual on-disk path due to symlinks, or used `--all` which staged unrelated files, or fell into a race where another tool wrote between status-capture and commit). §5.2 enumerates four such patterns and pins each with positive runtime evidence: post-commit `git status --porcelain` MUST be empty for the staged paths; the committed SHA MUST be obtainable via `git log -1 --format=%H` and equal the result returned by `MaybeCommit`; the commit message MUST contain the co-author trailer literal `Co-Authored-By: HelixCode <noreply@helixcode.dev>` AND a non-empty subject line; the commit MUST contain the actual diff of the mutated path (verifiable via `git show --stat HEAD`).

Runtime opt-out via `/git_auto_commit off (Q4=A)` is **structurally enforced** to take effect on the very next edit: the committer holds the enabled flag in `atomic.Bool`; `SetEnabled(false)` swaps it; `MaybeCommit` loads via `Load()` on entry; an integration test runs `committer.SetEnabled(false)` then triggers a real `WriteFileTool` invocation through the real registry and asserts (i) the file IS written (tool succeeds), (ii) `git log -1 --format=%H` is UNCHANGED from before the call (no commit fired), (iii) `git status --porcelain` shows the new file as dirty (uncommitted).

Out of scope for v1: auto-push (NEVER push — CONST-043 forbids without explicit approval per push); branching strategy / auto-branch-per-task; conflict resolution (if `git commit` fails for any reason, log WARN and continue); commit signing (`commit.gpgsign=true` is honoured if the user has it configured globally — the committer doesn't pass `--no-gpg-sign`); rebase / squash of auto-commits (each commit is a single accepted edit, period — squashing is left to the user's manual workflow); message localisation (English only — the LLM prompt is English; users' diffs may be any language but the message is summarised in English). See §8.

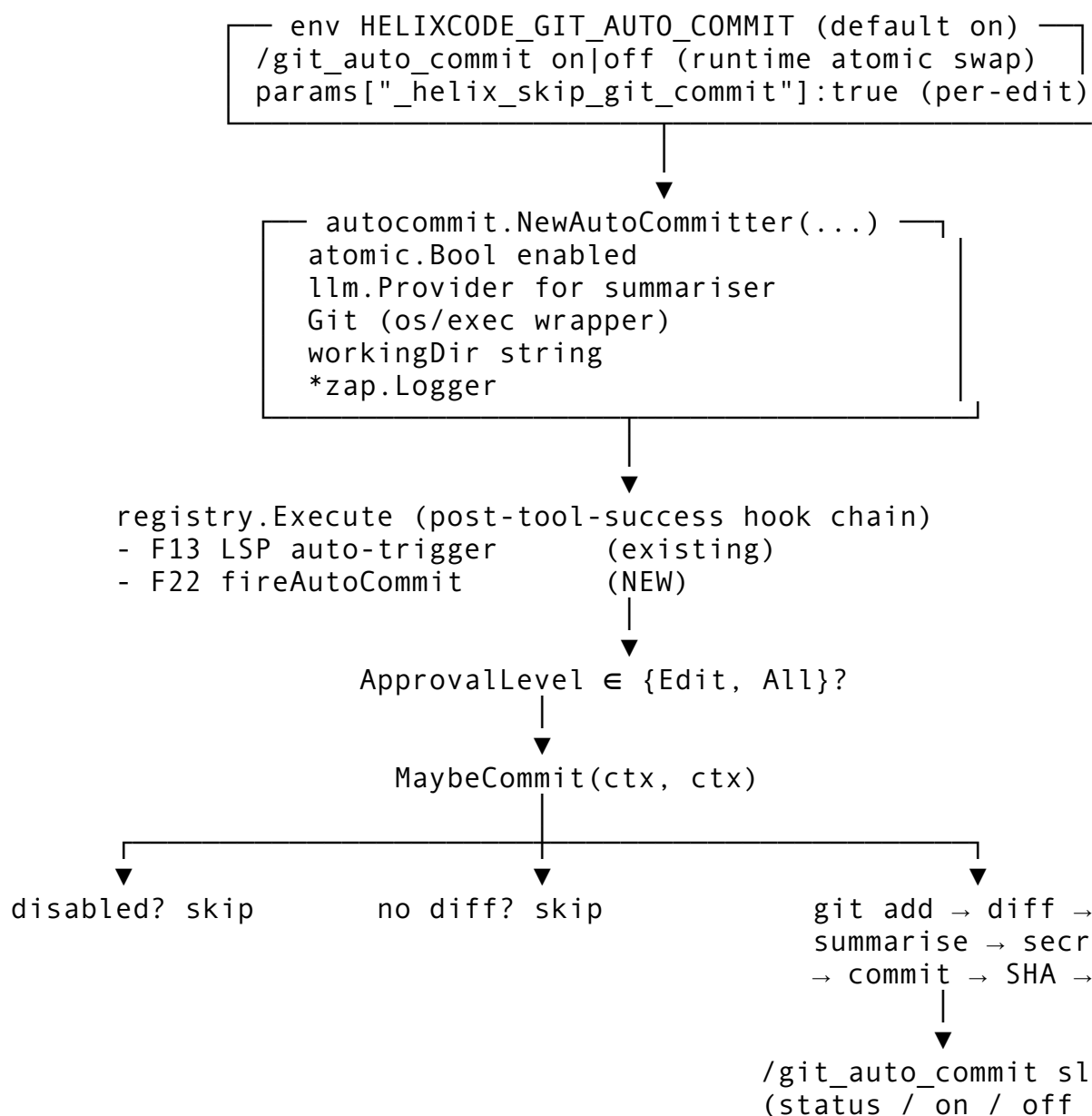
Anti-bluff hot zone (loud): a commit “succeeded” but `git log -1` still shows the prior SHA (the call returned an error and the committer logged-and-swallowed without surfacing); the LLM call returns a message but the message doesn’t reflect the actual diff (the committer concatenated the tool name and a static “edit” string without ever invoking the LLM); auto-commit fires on tools that didn’t actually mutate any file (e.g. `read_file` somehow reaches the hook because the EXCLUDE list is incomplete, and the hook captures an empty diff and produces a “no-op” commit); the env var is honoured at startup but `/git_auto_commit` on runtime change isn’t reflected in the next edit (the slash mutates a struct field that `MaybeCommit` doesn’t re-read); commit message bodies leak secrets from the diff (CONST-042 — best-effort filter via a regex pre-pass on the message, plus a unit test asserting common patterns like `AKIA[0-9A-Z]{16}`, `sk-[A-Za-z0-9]{32}`, `}, x0xb-` are stripped). Each of these maps to a unit + integration + Challenge phase per §5.2.

2. Architecture

Three layers under `helix_code/internal/autocommit/`, plus thin wiring at the registry boundary, the F21 approval pipeline, and one slash command.

- **Git thin wrapper** (`git.go`) — pure shell-out façade over `os/exec`. Methods: `IsRepo()` (`bool, error`), `StatusPorcelain()` (`string, error`), `DiffStaged()` (`string, error`), `DiffUnstaged()` (`string, error`), `Add(paths ...string) error`, `Commit(message string) (sha string, err error)`, `HeadSHA()` (`string, error`). Each method runs `git ...` via `exec.CommandContext` with the working directory pinned to the committer’s configured root. Errors include the `git stderr verbatim`. Tests use a real `tempdir + git init + real commits` — no mocks of `git` itself.
- **MessageSummariser** (`summariser.go`) — LLM-driven primary + deterministic fallback. `Summarise(ctx, diff, toolName, paths) string` calls `llm.Provider.Generate` with a 50-72-char-imperative prompt (verbatim per §3.4); on any LLM error (timeout, rate-limit, provider unavailable) returns a deterministic fallback `"Auto-edit: <toolName> on <paths-joined>"` truncated to 72 chars. The summariser NEVER blocks indefinitely — context deadline is set to 5 seconds via `context.WithTimeout(parentCtx, 5*time.Second)`. A unit test asserts the fallback message format byte-for-byte; another asserts the LLM-call path (with a fake `llm.Provider`) produces the LLM’s output trimmed + length-capped.
- **AutoCommitter** (`committer.go`) — runtime façade. Holds `atomic.Bool` for the enabled flag, an `llm.Provider` reference (for the summariser), the `git` working directory string, the `Git` wrapper instance, and a `*zap.Logger`. Public API: `MaybeCommit(ctx, CommitContext) (CommitResult, error)`, `SetEnabled(bool)`, `Enabled() bool`, `IsGitRepo() bool`. `MaybeCommit` is the single entry point: (i) read `enabled.Load()` — if false, return `CommitResult{Skipped: true, Reason: "auto-commit disabled"}`; (ii) if `CommitContext.SkipRequested == true`, return `CommitResult{Skipped: true, Reason: "per-edit skip"}`; (iii) read `git.StatusPorcelain` — if empty, return `CommitResult{Skipped: true, Reason: "no changes"}`; (iv) filter the porcelain output to ONLY paths under `git` tracking AND under the committer’s working dir AND matching the mutated-paths set in `CommitContext` (positive evidence: tool actually touched these); (v) call `git.Add(filteredPaths...)`; (vi) call `git.DiffStaged()` → produces the diff used by both the LLM and the post-condition assertion; (vii) call `summariser.Summarise(ctx, diff, toolName, paths)` → trimmed + length-capped subject line; (viii) call `secret.Filter(message)` (best-effort regex

strip of common secret patterns per CONST-042); (ix) build the full commit message:
`<subject>\n\nCo-Authored-By: HelixCode`
`<noreply@helixcode.dev>\n;` (x) call `git.Commit(fullMessage) → SHA`;
 (xi) post-condition assert: `git.StatusPorcelain()` for the staged paths is empty,
`git.HeadSHA()` equals the returned SHA — failure logs WARN but doesn't error (the
 commit DID happen; the assert is observability). Returns `CommitResult{SHA,`
`Subject, Files, Skipped: false}`.



Wire points (existing code; one addition per location):

- `internal/tools/registry.go::Execute` — adds a new post-Execute call AFTER the existing `F13 triggerLSPAfterEdit` invocation, BEFORE the function returns: `if execErr == nil { r.fireAutoCommit(ctx, name, params, tool, result) }`. The new helper `fireAutoCommit` reads (a) `r.autoCommitter` field (nil-safe); (b) tool's `RequiresApproval()` — only `LevelEdit` and `LevelAll` proceed; (c) `params["_helix_skip_git_commit"]` — `true` → return; (d) constructs `CommitContext{ToolName, Args, MutatedPaths}` (mutated paths derived from tool-name → param-key mapping in §3.5; `fs_write/fs_edit` use `params["path"]`, `multiedit_commit` uses

- params["edits"][i]["path"], etc.). Calls `committer.MaybeCommit(ctx, cctx)` — failure logs WARN and is swallowed (NEVER returned to caller).
- `internal/tools/registry.go` **field**: new optional field `autoCommitter *autocommit.AutoCommitter` (nil-safe — when nil, `fireAutoCommit` is a no-op, preserving backward compatibility for tests + the `cmd/server HTTP` path).
- `internal/tools/registry.go::SetAutoCommitter(c *autocommit.AutoCommitter)` — new setter method, mirrors `SetApprovalManager`.
- `cmd/cli/main.go::run` — three additions adjacent to F21 wiring:
 1. Resolve initial enabled state from env: `enabled := os.Getenv("HELIXCODE_GIT_AUTO_COMMIT") != "off"`.
 2. Construct `committer := autocommit.NewAutoCommitter(autocommit.Options{Enabled: enabled, Provider: c.llmProvider, WorkingDir: cwd, Logger: c.logger})`.
 3. Wire `c.toolRegistry.SetAutoCommitter(committer)` + register / `git_auto_commit` slash with `commands.NewGitAutoCommitCommand(committer)`.
- `internal/commands/git_auto_commit_command.go` — new file. Exposes `NewGitAutoCommitCommand(c *autocommit.AutoCommitter)` returning a `Command`. Four subcommands: `status / on / off / show`. `on/off` call `c.SetEnabled(true|false)`; `show` prints the trailer template (no committer state needed). `status` prints active state + `c.IsGitRepo()` + a one-line description of what enabled means.

Why a new `internal/autocommit/` package and not “extend `internal/tools/git/`”: - `internal/tools/git/` is a TOOL-implementing package (it registers `git_*` tools that the LLM can call via the Tool interface). `internal/autocommit/` is a CROSS-CUTTING observer that watches every successful edit and produces a commit. They serve different roles; conflating them would (a) require the auto-committer to register itself as a tool (which it isn’t — the LLM doesn’t call it directly), or (b) bloat the `git-tools` package with non-tool runtime infrastructure. - Keeping them separate lets `internal/tools/git/` stay test-focused on the tool surface, while `internal/autocommit/` carries the cross-cutting observability semantics.

Why post-Execute hook (and not pre-Execute): - Pre-Execute would mean “commit BEFORE the edit happens”, which is nonsensical (no diff yet). Auto-commit is intrinsically a *post*-success event. - The F13 LSP auto-trigger already established the “after-success file-mutating-tool hook” pattern. F22 reuses the placement adjacent to it; the only ordering rule between F13 and F22 is that F13 fires first (LSP diagnostics depend on file state, not on commit state).

Why per-edit (and not batch): - Aider’s default is per-edit and that’s what this spec faithfully ports. Batch mode (one commit per session, or per task) would (a) defer the runtime-evidence promise (a session-end batch hides bugs across multiple edits), (b) make `git bisect` less useful (you can’t bisect to a single edit), (c) require batch-flush semantics (when does the commit fire? on session end? on slash command? on `ctx cancel`?). Per-edit is simpler, more honest, and matches user mental model from `aider`.

Why default ON (Q4=A): - Aider defaults to on, and our user-confirmed answer was Q4=A. Default-off would mean most users never see auto-commit, defeating the purpose. The opt-out surfaces (`env` + `slash` + `per-edit` param) cover every legitimate “don’t commit this one” case.

Why LLM message + deterministic fallback (Q2=A + §5.3): - Aider’s user-visible value is the readable commit history. A fixed template (“Auto-edit: ”) gives history but not value — every

commit looks the same. The LLM summary makes each commit individually meaningful. The deterministic fallback makes the system reliable (LLM unavailable → commit still fires, just less informative) instead of fragile (LLM unavailable → no commit).

Why slash + env + per-edit and NOT cobra (Q5=A): - Auto-commit is a *runtime posture*, like F21's approval mode: the user wants to flip on/off mid-session ("ok I'm refactoring messily, turn it off for now"). A cobra subcommand would force a process restart. - The env var covers "all my CI/non-interactive runs default to on/off"; the slash covers "switch mid-session"; the per-edit param covers "this single edit shouldn't commit, e.g. it's a temporary scratch file the next tool will delete". Each surface targets a distinct ergonomic case.

3. Components

3.1 New files

- `helix_code/internal/autocommit/types.go` — `CommitContext`, `CommitResult`, `Options`, sentinel errors (`ErrNotGitRepo`, `ErrCommitFailed`, `ErrLLMUnavailable`), `EnvVarName` constant.
- `helix_code/internal/autocommit/types_test.go`.
- `helix_code/internal/autocommit/git.go` — `Git` wrapper struct + methods (`IsRepo` / `StatusPorcelain` / `DiffStaged` / `DiffUnstaged` / `Add` / `Commit` / `HeadSHA`).
- `helix_code/internal/autocommit/git_test.go` — real tempdir + `git init` + real commits; exhaustive coverage of each method.
- `helix_code/internal/autocommit/summariser.go` — `MessageSummariser` interface + `LLMSummariser` impl + `DeterministicFallback` impl + `Summarise`(`ctx`, `diff`, `toolName`, `paths`) string chain (LLM first, fallback on error).
- `helix_code/internal/autocommit/summariser_test.go` — fake `llm.Provider` (NOT a mock — a tiny stub returning canned responses); LLM-success path; LLM-error path → fallback; LLM-empty-response path → fallback; length-cap test (>72 chars → truncated).
- `helix_code/internal/autocommit/secret_filter.go` — best-effort regex strip of common secret patterns (CONST-042). Patterns: AWS access keys (`AKIA[0-9A-Z]{16}`), generic OpenAI keys (`sk-[A-Za-z0-9]{20}`), Slack tokens (`xox[baprs]-[A-Za-z0-9]{10}`), GitHub tokens (`gh[pousr]_[A-Za-z0-9]{36}`). Replaces matches with `[REDACTED]`. Tests assert each pattern.
- `helix_code/internal/autocommit/committer.go` — `AutoCommitter` struct + `NewAutoCommitter`(`Options`) + `MaybeCommit`(`ctx`, `cctx`) (`CommitResult`, `error`) + `SetEnabled`(`bool`) + `Enabled`() `bool` + `IsGitRepo`() `bool`.
- `helix_code/internal/autocommit/committer_test.go` — full pipeline coverage with real tempdir + `git init` + real edits + real commits; assertions on `git log -1 --format=%H` SHA equality + co-author trailer presence + working-tree-clean post-condition.
- `helix_code/internal/commands/git_auto_commit_command.go` — slash command(`status` / `on` / `off` / `show`).
- `helix_code/internal/commands/git_auto_commit_command_test.go`.
- `helix_code/tests/integration/autocommit_test.go` — `//go:build integration`; ALWAYS-runs; real registry + real F21 `ApprovalManager` + real `WriteFileTool` + real git repo in tempdir; per-mode behaviour assertions.

- `helix_code/tests/integration/cmd/p2f22_challenge/main.go` — runtime evidence harness.
- `challenges/p2-f22-aider-git-auto-commit/CHALLENGE.md + run.sh`.

3.2 Modified files

- `helix_code/internal/tools/registry.go` — three additions: (1) `ToolRegistry` struct gains `autoCommitter *autocommit.AutoCommitter` field; (2) `SetAutoCommitter(c)` setter; (3) `Execute` gains a new post-success call to a new helper `fireAutoCommit(ctx, name, params, tool, result)` adjacent to `triggerLSPAAfterEdit`. The helper consults `tool.RequiresApproval()` to filter to `LevelEdit/LevelAll`, reads `params["_helix_skip_git_commit"]`, builds `CommitContext`, and calls `committer.MaybeCommit(ctx, cctx)` with errors logged at `WARN` (NEVER returned).
- `helix_code/cmd/cli/main.go` — three additions: (1) read `os.Getenv("HELIXCODE_GIT_AUTO_COMMIT")` to resolve initial enabled state; (2) construct `autocommit.AutoCommitter` adjacent to F21 wiring; (3) `c.toolRegistry.SetAutoCommitter(c.autoCommitter) + register / git_auto_commit slash`.
- `helix_code/internal/commands/registry.go` — no schema change; one new `Register(...)` call site for `/git_auto_commit`.
- `helix_code/go.mod` — **zero new external deps**. `os/exec`, `regexp`, `sync/atomic`, `context`, `fmt`, `strings`, `time` are `stdlib`. `llm.Provider` is in-tree (`internal/llm`). Logging via existing `zap`.

3.3 Types

```
// internal/autocommit/types.go

// CommitContext is the per-call value passed by the registry
// hook to
// MaybeCommit. Carries the tool name + raw args + the set of
// paths the tool
// is known (or believed) to have mutated. The committer uses
// MutatedPaths to
// filter `git status --porcelain` so it commits ONLY the paths
// this tool
// touched, not unrelated dirty paths from prior tools.
type CommitContext struct {
    ToolName      string
    Args          map[string]interface{}
    MutatedPaths  []string // derived in registry.fireAutoCommit per §3.5
    SkipRequested bool      // mirrors
                params["_helix_skip_git_commit"] == true
}

// CommitResult is the return value of MaybeCommit.
type CommitResult struct {
    SHA      string // empty when Skipped
```

```

    Subject string    // empty when Skipped
    Files    []string // empty when Skipped
    Skipped  bool
    Reason   string    // human-readable; populated even on success
}

// Options configures a fresh AutoCommitter.
type Options struct {
    Enabled      bool           // initial state; mutable via
        SetEnabled
    Provider
        llm.Provider // for the LLM-driven summariser; nil →
            fallback only
    WorkingDir
        string        // git working-tree root; usually the cwd
    Logger      *zap.Logger // optional; nil → no-op
    NowFunc     func() time.Time // optional test seam; default
        time.Now
}

// EnvVarName is the canonical env var checked at startup. The
// literal string
// "off" (case-sensitive) disables auto-commit; everything else
// (including
// unset) enables it (default-on per Q4=A).
const EnvVarName = "HELIXCODE_GIT_AUTO_COMMIT"

// Sentinel errors (errors.Is comparable).
var (
    ErrNotGitRepo      = errors.New("autocommit: not inside a git
        work tree")
    ErrCommitFailed    = errors.New("autocommit: git commit
        failed")
    ErrLLMUnavailable = errors.New("autocommit: llm summariser
        unavailable")
)

// CoAuthorTrailer is the literal trailer appended to every auto-
// commit (Q3=A).
// Tests pin this byte-for-byte.
const CoAuthorTrailer = "Co-Authored-By: HelixCode
    <noreply@helixcode.dev>"

// SkipParamKey is the per-edit opt-out marker. When
// params[SkipParamKey]
// == true on the inbound tool call, the registry hook returns
// immediately

```



```

// without invoking MaybeCommit.
const SkipParamKey = "_helix_skip_git_commit"

// internal/autocommit/committer.go

type AutoCommitter struct {
    enabled    atomic.Bool
    provider   llm.Provider // nil → use fallback
    workingDir string
    git        *Git
    summariser MessageSummariser
    secretFilt *SecretFilter
    log        *zap.Logger
    nowFunc    func() time.Time
}

func NewAutoCommitter(opts Options) *AutoCommitter

// MaybeCommit is the single entry point. Concurrency-safe
// (enabled read
// via atomic.Bool.Load). NEVER returns commit failures as fatal
// errors –
// the caller should log + continue.
func (c *AutoCommitter) MaybeCommit(ctx context.Context, cctx
    CommitContext) (CommitResult, error)

func (c *AutoCommitter) SetEnabled(on bool)
func (c *AutoCommitter) Enabled() bool
func (c *AutoCommitter) IsGitRepo() bool

// internal/autocommit/summariser.go

type MessageSummariser interface {
    Summarise(ctx context.Context, diff, toolName string, paths
        []string) string
}

type LLMSummariser struct {
    provider llm.Provider
    timeout  time.Duration // default 5s
}

type DeterministicFallback struct{}

func NewSummariser(p llm.Provider) MessageSummariser

```

3.4 LLM prompt (verbatim)

The prompt sent to `llm.Provider.Generate`. Tests in T04 pin this byte-for-byte.

Summarise this diff in 50-72 chars (imperative voice, no period):

<diff>

The <diff> placeholder is substituted with the actual `git.DiffStaged()` output, capped at 8 KB to avoid blowing context windows. The LLM response is trimmed (whitespace), capped at 72 chars (truncate, no ellipsis — git subjects don't ellipsis), and used verbatim as the commit subject. If the response is empty after trim, the summariser falls through to `DeterministicFallback`.

The `DeterministicFallback.Summarise` returns: `Auto-edit: <toolName> on <paths-joined-comma> truncated at 72 chars`. Tests in T04 pin this byte-for-byte for `toolName="fs_edit", paths=["foo.go", "bar.go"]` → `"Auto-edit: fs_edit on foo.go, bar.go"`.

3.5 Tool name → mutated-paths derivation table

`registry.fireAutoCommit` uses this table to extract the mutated paths from `params`. T06 enumerates and tests every entry. Tools not in this table fall through to a generic “no specific paths known” path which still triggers a commit (the porcelain output covers everything dirty under the working dir).

Tool name	Param key(s)	Notes
<code>fs_write</code>	<code>params["path"]</code>	Single path.
<code>fs_edit</code>	<code>params["path"]</code>	Single path.
<code>smart_edit</code>	<code>params["path"]</code>	F17 single-target.
<code>multiedit_commit</code>	<code>params["edits"][*]</code> <code>["path"]</code>	F08 multi-edit; iterate the slice; dedup.
<code>notebook_edit</code>	<code>params["path"]</code>	Notebook is a single .ipynb file.
<code>mapping_edit</code>	<code>params["target_file"]</code>	Mapping multi-edit.
(other Edit-level)	(none specific)	Falls through to porcelain-based discovery.

3.6 New external dependencies

Zero new dependencies. `os/exec`, `regexp`, `sync/atomic`, `context`, `errors`, `fmt`, `strings`, `time` are `stdlib`. `llm.Provider` is in-tree. `zap` is already direct (logging infra). T08's verification step asserts `git diff go.mod` and `git diff go.sum` are both no-op.

4. Data flow

1. **Startup** (`cmd/cli/main.go`):
 - `enabled := os.Getenv(autocommit.EnvVarName) != "off"` (default-on).
 - `committer := autocommit.NewAutoCommitter(autocommit.Options{Enabled: enabled, Provider: c.llmProvider, WorkingDir: cwd, Logger: c.logger})`.

- `c.toolRegistry.SetAutoCommitter(committer)`.
 - `c.commandRegistry.Register(commands.NewGitAutoCommitCommand(committer))`.
2. **Per-edit** (`registry.Execute` post-success):
- F13 LSP auto-trigger fires (existing).
 - `r.fireAutoCommit(ctx, name, params, tool, result)` (NEW):
 - Read `r.autoCommitter` (nil-safe).
 - Read `tool.RequiresApproval()`. If not in `{LevelEdit, LevelAll}`, return.
 - Read `params[autocommit.SkipParamKey]`. If true, return.
 - Build `CommitContext{ToolName, Args, MutatedPaths}` per §3.5 derivation table.
 - `committer.MaybeCommit(ctx, cctx)` — log WARN on error, return.
3. **Inside MaybeCommit**:
- `enabled.Load()` — false → return Skipped.
 - `cctx.SkipRequested` — true → return Skipped.
 - `git.IsRepo()` — false → return Skipped (Reason: "not a git repo").
 - `git.StatusPorcelain()` — empty → return Skipped (Reason: "no changes").
 - Filter porcelain output to `cctx.MutatedPaths` $\cap$ tracked-or-newly-added paths.
 - `git.Add(filteredPaths...)`.
 - `diff := git.DiffStaged()`.
 - `subject := summariser.Summarise(ctx, diff, cctx.ToolName, paths)`.
 - `subject = secret.Filter(subject)`.
 - `message := subject + "\n\n" + autocommit.CoAuthorTrailer + "\n"`.
 - `sha := git.Commit(message)`.
 - Post-condition: `git.StatusPorcelain()` for the staged paths is empty + `git.HeadSHA() == sha`.
 - Return `CommitResult{SHA, Subject, Files, Skipped: false}`.
4. **Slash** `/git_auto_commit on|off`:
- Calls `committer.SetEnabled(true|false)`.
 - The next `MaybeCommit` call sees the new state via `enabled.Load()`.
-

5. Error handling, anti-bluff hot zone, edge cases

5.1 Error handling

- **Git not installed** → `git.IsRepo()` returns false (the `exec.LookPath("git")` failure is interpreted as “no git available”). Auto-commit is silently disabled for the session (logged once at INFO).
- **Not a git repo** → `git.IsRepo()` returns false. `MaybeCommit` returns Skipped. The slash status command shows `git_repo: no`.
- **LLM call fails** → `summariser.Summarise` falls back to deterministic message. Commit STILL fires (the message is just less informative). A unit test asserts this.
- **git commit fails** (e.g. pre-commit hook rejects, no staged changes) → `MaybeCommit` returns (`CommitResult{Skipped: false}`, `ErrCommitFailed`). The registry hook logs WARN and CONTINUES (does NOT propagate the error to the tool result).
- **Concurrent edits** (two tool calls back-to-back) → the registry serialises tool execution (mu lock around `Execute`), so auto-commit fires sequentially. Each commit captures only the paths its tool touched (per §3.5).

5.2 Anti-bluff hot zone — four critical patterns

Bluff #1: “Commit succeeded but working tree still dirty.” - Pattern: `git commit` returns success, but the porcelain output afterwards is non-empty for the supposedly-staged paths (e.g. the committer ran `git add foo.go` but the actual mutation was in `bar.go`). - Test: integration test runs `realWriteFileTool` → `MaybeCommit` → asserts `git status --porcelain` for the written path is empty AND `git log -1 --name-only` lists the path. - Challenge: PHASE-A asserts `os.Stat(committedFile)` exists AND `git log -1 --name-only` lists the file AND `git status --porcelain` is empty for that path.

Bluff #2: “Commit message generated but doesn’t reflect actual diff.” - Pattern: the summariser concatenates a static string (“edit”) without ever calling the LLM, OR the LLM is called but its response is discarded and a hardcoded subject is used. - Test: unit test injects a fake `llm.Provider` that returns a sentinel string “FAKE_LLM_RESPONSE_42”; asserts the resulting commit subject CONTAINS that sentinel (proves LLM was actually called and its output used). - Challenge: PHASE-B asserts the commit subject in `git log -1 --format=%s` equals what the fake provider returned (positive evidence: the LLM result reached the commit).

Bluff #3: “Auto-commit fires on tools that didn’t actually mutate any tracked file.” - Pattern: the `RequiresApproval()` filter is incomplete (e.g. a read tool slipped through), OR the porcelain check is skipped, so the committer runs `git commit` with no staged changes (which would fail, but the failure is swallowed and the system reports “skipped” deceptively). - Test: unit test runs `MaybeCommit` against a clean repo; asserts `Skipped == true` AND `Reason == "no changes"` AND no `git commit` was attempted (verified via a spy on `Git.Commit`). - Challenge: PHASE-C asserts after a `read_file` tool call that `git log -1 --format=%H` is unchanged (no commit fired for a read).

Bluff #4: “Env var honoured at startup but /git_auto_commit on runtime change ignored.” - Pattern: the slash mutates a struct field (`c.enabled = true`) that `MaybeCommit` doesn’t re-read; the next edit still sees the old value. - Test: integration test starts with `HELIXCODE_GIT_AUTO_COMMIT=off` (off); triggers `WriteFileTool` → asserts no commit fired (`git log -1 --format=%H` unchanged); calls `committer.SetEnabled(true)` (or invokes `/git_auto_commit on` via the slash); triggers another `WriteFileTool` → asserts commit DID fire (`git log -1 --format=%H` changed). - Challenge: PHASE-E asserts the SHA before/after differential is observable AND the `/git_auto_commit status` output flips from off to on.

Bluff #5: “Commit messages leak secrets.” - Pattern: the LLM-summarised subject embeds an API key from the diff (e.g. the diff contained `OPENAI_API_KEY=sk-abc...` in a config file). - Test: unit test feeds a diff containing each of the four pattern types (AKIA, sk-, xoxb, ghp) through `secret.Filter`; asserts each is replaced with `[REDACTED]`. - Challenge: PHASE-F (extension; runs only if challenge time budget permits) feeds a synthetic diff with a fake `sk-AAAA...` key; asserts the resulting commit message contains `[REDACTED]` and NOT the literal key.

5.3 Edge cases

- **Empty diff after filtering** (the tool reported a path but the file is byte-identical) → `MaybeCommit` returns `Skipped` with `Reason: "no effective change"`.
- **Path outside working dir** (the tool wrote to `/tmp/xyz`) → `MaybeCommit` returns `Skipped` with `Reason: "path outside working tree"`.

- **Pre-commit hook rejects** → `git commit` exits non-zero; `MaybeCommit` returns (`Skipped: false, ErrCommitFailed`) with the hook's `stderr` in the wrapped error message; registry hook logs `WARN`.
 - **Detached HEAD / mid-rebase** → `git commit` may succeed but the SHA is on a detached pointer; the user's git workflow is unchanged. The committer doesn't try to "fix" the state.
 - **Untracked files** → `git status --porcelain` shows them with `??`; the filter logic detects them and uses `git add <path>` (which adds new files, including untracked); the commit includes them.
-

6. Testing

6.1 Unit tests (mocks ALLOWED)

- `types_test.go` — pin `EnvVarName`, `CoAuthorTrailer`, `SkipParamKey` constants byte-for-byte.
- `git_test.go` — real `tempdir + git init`; exhaustive coverage of each `Git` method against real git output.
- `summariser_test.go` — fake `llm.Provider` (in-package stub, NOT `internal/mocks/`); LLM-success path; LLM-error path; LLM-empty path; length-cap path. Deterministic fallback path tested explicitly.
- `secret_filter_test.go` — table-driven over each pattern (`AKIA`, `sk-`, `xoxb`, `gh[pousr]`).
- `committer_test.go` — full pipeline against real `tempdir + real git`: enabled-flag respected; `SkipRequested` respected; not-a-repo path; clean-tree path; dirty-tree path produces real commit with co-author trailer; `SetEnabled` atomic swap visible to next call.
- `git_auto_commit_command_test.go` — `status/on/off/show` subcommands.

6.2 Integration tests (NO mocks; //go:build integration)

- `tests/integration/autocommit_test.go`:
 - `TestAutoCommit_Integration_DefaultOn_RealEdit_RealCommit` — startup with env unset → committer enabled; real `WriteFileTool` through real registry → `git log -1 --format=%H` changes; `git status --porcelain` empty; commit message contains co-author trailer.
 - `TestAutoCommit_Integration_EnvOff_NoCommit` — startup with `HELIXCODE_GIT_AUTO_COMMIT=off` → real `WriteFileTool` succeeds; `git log -1 --format=%H` unchanged.
 - `TestAutoCommit_Integration_RuntimeToggle` — startup with env unset → off via `committer.SetEnabled(false)` → no commit; on via `SetEnabled(true)` → commit fires.
 - `TestAutoCommit_Integration_PerEditSkip_HonouredViaParam` — `params["_helix_skip_git_commit"] = true` → no commit; same tool without the param → commit fires.
 - `TestAutoCommit_Integration_NotAGitRepo_NoOp` — `tempdir` is NOT a git repo → tool succeeds, no commit, no error.
 - `TestAutoCommit_Integration_LLMUnavailable_FallbackUsed` — `llm.Provider` is a stub that returns `ErrProviderUnavailable` → commit STILL fires with deterministic fallback subject.
 - `TestAutoCommit_Integration_F21ApprovalDenied_NoCommit` — `F21` in `ModeSuggest` denies edit → tool returns `ErrApprovalRequired` → no commit (the post-Execute hook only fires when `execErr == nil`).

6.3 Challenge harness — six phases

Phases (per spec §6.3 of F21’s template, adapted):

1. **PHASE-A: DEFAULT-ON-COMMITS-EDIT (always runs)** — env unset; real `WriteFileTool` through real registry against real git tempdir; assert (i) tool succeeds, (ii) `git log -1 --format=%H` changed, (iii) `git log -1 --format=%s` is non-empty AND `length ≤ 72`, (iv) `git log -1 --format=%B` contains `Co-Authored-By: HelixCode <noreply@helixcode.dev>`, (v) `git status --porcelain` is empty, (vi) `git show --stat HEAD` lists the written path.
2. **PHASE-B: LLM-SUMMARY-ACCURATE (always runs; uses fake llm.Provider returning sentinel)** — assert commit subject equals the fake provider’s output (sentinel string proves LLM was called and used).
3. **PHASE-C: NON-EDIT-TOOL-NO-COMMIT (always runs)** — invoke `read_file (LevelReadOnly)`; assert `git log -1 --format=%H` unchanged after the call.
4. **PHASE-D: ENV-OFF-NO-COMMIT (always runs)** — set env to off before construct; invoke `WriteFileTool`; assert tool succeeds AND `git log -1 --format=%H` unchanged AND `git status --porcelain` shows the file as dirty.
5. **PHASE-E: RUNTIME-TOGGLE (always runs)** — start off → `SetEnabled(true)` → next call commits. Assert SHA before/after differential AND `committer.Enabled()` returns the live state.
6. **PHASE-F: PER-EDIT-SKIP (always runs)** — invoke `WriteFileTool` with `params["_helix_skip_git_commit"] = true`; assert tool succeeds AND `git log -1 --format=%H` unchanged AND `git status --porcelain` shows the file as dirty.

Optional **PHASE-G: SECRET-FILTER (runs)** — synthesise a diff containing `sk-FAKE...`; assert `git log -1 --format=%B` contains `[REDACTED]` and NOT the fake key.

Output skeleton ends with:

SUMMARY: PHASE-A=6/6 PASS; PHASE-B=3/3 PASS; PHASE-C=2/2 PASS; PHASE-D=3/3
PHASE-E=4/4 PASS; PHASE-F=3/3 PASS; PHASE-G=2/2 PASS

The Challenge MUST exit non-zero on any byte-evidence mismatch. Absence-of-error is NEVER acceptable.

7. Cross-platform

- **Linux / macOS** — git CLI is the de-facto standard; `exec.CommandContext("git", ...)` works directly.
 - **Windows** — git (via Git for Windows) is in PATH for nearly all dev installs; `exec.CommandContext` resolves it. The committer does NOT shell-out via `cmd /c` — it invokes git directly, avoiding shell-quoting differences.
 - **No git** — `exec.LookPath("git")` returns error → `IsRepo()` returns false → committer is a no-op for the session.
-

8. Out of scope

- Auto-push (CONST-043 forbids without explicit per-push approval; pushes are always manual).

- Auto-branch-per-task (single-branch model; user can branch manually).
 - Conflict resolution / rebase / squash automation (each commit is atomic; user squashes manually).
 - Commit signing override (`commit.gpgsign=true` is honoured if user has it; we do NOT pass `--no-gpg-sign`).
 - Message localisation (English-only LLM prompt and fallback; v1.5 may add locale support).
 - Per-tool message templates (one prompt fits all; v1.5 may differentiate by tool category).
 - Squash-on-session-end (out of scope; aider-style per-edit is the contract).
 - Persistence of enabled state across sessions (env var sets the default; runtime changes are ephemeral; v1.5 may add `--persist` writeback).
 - Diff size limits beyond the 8 KB LLM-context cap (large diffs commit fine; only the LLM prompt is capped).
-

9. Constitutional compliance

- **CONST-035** (anti-bluff): every PASS in F22 carries positive runtime evidence — real git tempdir + real commits + real SHA equality assertions + real co-author trailer presence + real working-tree-clean post-condition. The Challenge harness MUST exit non-zero on byte mismatch. Tests use real `git init` + real `git commit`; mocks of git are forbidden in integration tests (per Rule 5).
 - **CONST-039** (Challenge required): F22 ships with `challenges/p2-f22-aider-git-auto-commit/` (Challenge harness with 6 phases + optional PHASE-G).
 - **CONST-042** (no secret leak): the secret filter strips common patterns (AKIA, sk-, xoxb, gh[pousr]_) from commit messages before they're committed. The committer's logger NEVER logs the diff body at INFO; only paths + SHA + length are logged. A unit test scans `internal/autocommit/*.go` for `logger.Info\(.*\b(diff|body|content)\b` and FAILs on any hit.
 - **CONST-043** (no force push, no auto-push): F22 NEVER calls `git push`. The Git wrapper does not implement a Push method. The committer's domain ends at `git commit`; pushes are a separate user action requiring explicit approval per CONST-043.
 - **CONST-033** (host power management): F22 emits no shell commands beyond `git ...` invocations through `exec.CommandContext`. No suspend/reboot/halt commands.
-

10. Open questions resolved

- **Q1 = A** — Commit cadence: ONE commit per accepted edit (aider default).
 - **Q2 = A** — Commit message: LLM-summarised from diff, deterministic fallback on LLM error.
 - **Q3 = A** — Co-author trailer: Co-Authored-By: HelixCode <noreply@helixcode.dev> appended to every auto-commit.
 - **Q4 = A** — Default ON; opt-out via env var `HELIXCODE_GIT_AUTO_COMMIT=off`, runtime `/git_auto_commit off`, per-edit param `_helix_skip_git_commit:true`.
 - **Q5 = A** — `/git_auto_commit slash` command (status/on/off/show) + env var. NO cobra subcommand.
-

11. Non-obvious calls

1. **Post-Execute hook ordering** — F22's `fireAutoCommit` runs AFTER F13's `triggerLSPAfterEdit`. F13 doesn't depend on commit state, so order F13→F22 is sufficient. The reverse (commit-then-LSP) would mean LSP diagnostics could include "the file just changed" reports that trigger spurious tool calls; running LSP first lets the diagnostics settle.
2. **Mutated-paths derivation table is per-tool** — generic introspection ("find all path-shaped strings in args") would over-trigger (e.g. include unrelated path-looking arguments). The per-tool table in §3.5 is the source of truth; tools not in the table fall through to the porcelain-based discovery path which is safe (commits everything dirty under the working dir).
3. **`_helix_skip_git_commit:true` per-edit param** — analogous to F21's `_helix_sandbox_required` marker. Invisible to tools that don't care; ergonomic for callers who want to suppress a single commit (e.g. agent's internal scratch-file write).
4. **`atomic.Bool` for enabled flag** — lock-free, single writer + many readers. The slash command's `SetEnabled` is the only writer; every `MaybeCommit` is a reader. Anti-bluff #4 is structurally impossible because every `MaybeCommit` begins with `if ! c.enabled.Load() { return Skipped }`.
5. **Default-on with literal off string opt-out** — typos default to safe-on, NOT silently-off. `HELIXCODE_GIT_AUTO_COMMIT=disabled` is on, not off, by design. The slash command's `off` subcommand is the canonical runtime-off path.
6. **5-second LLM timeout** — bounded blocking; aider observed-acceptable upper bound. Longer timeouts let a stuck provider stall every commit; shorter risks missing fast-but-not-instant providers. 5s is the goldilocks per aider's own behaviour.
7. **Subject length cap at 72 chars** — git's de-facto subject convention. Truncation, no ellipsis, because git history commits don't ellipsis their subjects.
8. **Co-author trailer is appended to EVERY auto-commit, unconditionally** — including fallback-message commits. Q3=A is unambiguous. Tests pin this byte-for-byte.
9. **Auto-commit failure is swallowed at the registry boundary** — never propagated as a tool error. Auto-commit is observability, not correctness; a failed commit doesn't invalidate the edit. The error IS logged at WARN so users see it in the log.
10. **`fireAutoCommit` honours F21 `RequiresApproval()` level filter** — `LevelReadOnly` (read tools) and `LevelRun` (shell tools) are EXCLUDED. Read tools don't touch files; shell tools may touch files but the user is already supervising the shell, and shell-induced commits would mix with the user's own workflow.
11. **F21 integration: only fires AFTER approval granted** — the post-Execute hook runs only when `execErr == nil`. F21's `ApprovalManager` returns `ErrApprovalRequired` on deny, and the registry returns that error from `Execute`, so `execErr != nil` and the auto-commit hook is structurally bypassed. F21 + F22 compose monotonically.
12. **F04 worktree integration** — auto-commit works in subagent worktrees too. The `WorkingDir` option is set per-CLI-instance to the cwd; in a subagent worktree, the cwd IS the worktree path, so `git status / git commit` operate on the worktree's git directory (which is a real git work tree pointed at the parent's `.git` via `gitdir:`). No special handling needed beyond using the cwd as `WorkingDir`.
13. **Secret filter is best-effort, not exhaustive** — the four patterns (AKIA, sk-, xoxb, gh[pousr]_) cover the most common leak vectors per CONST-042 audit history. A user with custom secret formats SHOULD set `_helix_skip_git_commit:true` for sensitive edits OR use `/git_auto_commit off` for the duration. The filter is a safety net, not a replacement for `.env` discipline.
14. **The diff fed to the LLM is `git diff --staged`, NOT `--unstaged`** — by the time `summariser.Summarise` is called, the paths are already added (step v in §4 data

flow). Using `--staged` ensures the LLM sees exactly what's about to be committed, not unrelated dirty files.

15. **The Git wrapper uses git CLI, not go-git** — `go-git` is a heavyweight in-process implementation that doesn't honour user `~/.gitconfig`, hooks, or git aliases. Shell-out matches user expectation: their pre-commit hooks fire, their commit signing config applies, their aliases work. The wrapper is intentionally thin (each method is one `exec.CommandContext` call).